

B.TECH FIRST YEAR · PROGRAMMING FOR PROBLEM SOLVING

POINTERS & User Defined Data Types

Unit IV – Complete Study Module

A comprehensive, exam-oriented reference covering Pointers, Pointer Arithmetic, Arrays, Structures, Unions, and Memory Layouts in C Language

Subject	Year	Topic Count
C Programming	B.Tech I Year	14 Sections

TABLE OF CONTENTS

PART A — POINTERS

01	Introduction to Pointers	3
02	Address (&) and Dereferencing (*) Operators	4
03	Pointer Declaration and Initialization	5
04	Pointer Arithmetic	6
05	Pointers and Arrays	7
06	Array Manipulation using Pointers	8

PART B — USER DEFINED DATA TYPES

07	Introduction to User Defined Data Types	9
08	Structures in C	10
09	Structure Operations	11
10	Arrays of Structures	12
11	Unions in C	13
12	Structure vs Union	14

PART C — EXAM PREPARATION

13	Questions & Answers	15
14	Quick Revision Sheet & Textbook References	19

Introduction to Pointers

1.1 Definition

A **pointer** is a variable that stores the *memory address* of another variable. Instead of holding a data value directly, a pointer holds the location (address) where data is stored in memory. Pointers are one of the most powerful and distinctive features of the C programming language.

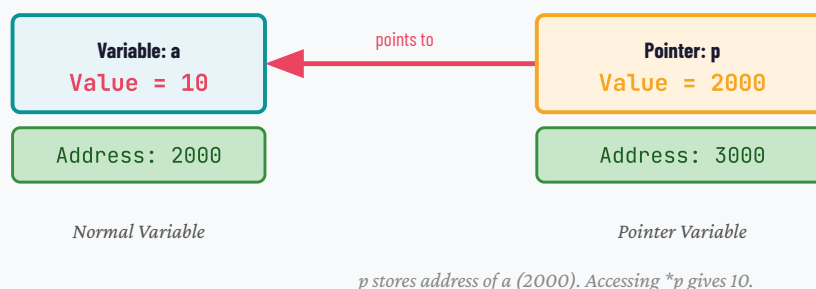
1.2 Why Pointers?

- Enable direct manipulation of memory addresses
- Facilitate efficient array and string handling
- Allow dynamic memory allocation (malloc, calloc)
- Enable passing large data structures to functions efficiently
- Form the foundation of linked lists, trees, and graphs
- Reduce time and space complexity in many algorithms

1.3 Memory Model in C

Every variable in C occupies memory. The system assigns each variable a unique address. When we declare `int a = 10;`, the value 10 is stored at some address, say 2000. A pointer variable can be declared to store this address 2000.

FIG 1.1 - POINTER VARIABLE REPRESENTATION IN MEMORY



1.4 Advantages of Pointers

- **Efficiency:** Pass-by-reference avoids copying large data
- **Dynamic Memory:** Runtime memory allocation using heap
- **Flexibility:** Arrays and strings handled efficiently
- **Data Structures:** Foundation of linked lists, trees

1.5 Limitations of Pointers

- Uninitialized pointers cause *dangling pointer* bugs
- Memory leaks if dynamic memory is not freed
- Segmentation fault if pointer goes out of bounds
- Difficult to debug pointer-related errors

Key Fact: Every pointer variable itself occupies memory — typically 4 bytes on 32-bit systems and 8 bytes on 64-bit systems, regardless of the data type it points to.

Address (&) and Dereferencing (*) Operators

2.1 Address-of Operator (&)

The **address-of operator (&)** returns the memory address of a variable. It is a unary operator applied before a variable name. The result is the address in hexadecimal format (e.g., 0x7fff2000).

Syntax: `&variable_name`

Example: `int a = 10; printf("%p", &a); // prints address of a`

2.2 Dereferencing Operator (*)

The **dereference operator (*)** — also called the *indirection operator* — accesses the value stored at the address contained in a pointer. It "goes to" the address pointed by the pointer and retrieves the value there.

Syntax: `*pointer_variable`

Example: `int *p = &a; printf("%d", *p); // prints 10`

FIG 2.1 - DEREFERENCING CONCEPT DIAGRAM



2.3 Step-by-Step Working

1. Declare `int a = 10;` — variable 'a' is stored at address 2000 with value 10
2. Declare `int *p;` — pointer p can hold address of int
3. Assign `p = &a;` — p now holds 2000 (address of a)
4. `*p` = dereference p → go to address 2000 → read value = 10

5. `*p = 20;` — changes value at address 2000, so a becomes 20

C PROGRAM — ADDRESS AND DEREFERENCE OPERATORS

```
#include <stdio.h>

int main() {
    int a = 10;
    int *p = &a;      // p holds address of a

    printf("Value of a      = %d\n", a);
    printf("Address of a    = %p\n", &a);
    printf("Value in p      = %p\n", p);
    printf("Value via *p    = %d\n", *p);

    *p = 50;           // modifying a through pointer
    printf("New value of a = %d\n", a); // 50

    return 0;
}
```

Output:

Value of a = 10 | Address of a = 0x7fff2000 | Value in p = 0x7fff2000 | Value via *p = 10 | New value of a = 50

Pointer Declaration and Initialization

3.1 Pointer Declaration

A pointer must be declared with a data type that tells the compiler what type of variable it will point to. The **asterisk (*)** is used in the declaration to indicate a pointer variable.

Syntax: `data_type *pointer_name;`

Examples:

```
int *p; // pointer to integer
float *fp; // pointer to float
char *cp; // pointer to character
double *dp; // pointer to double
```

3.2 Pointer Initialization

After declaration, a pointer must be initialized before use. An uninitialized pointer is called a *wild pointer* and using it causes undefined behavior.

POINTER DECLARATION AND INITIALIZATION

```
int    a = 100;
float  b = 3.14;
char   c = 'X';

int    *ip = &a;    // ip points to integer a
float  *fp = &b;    // fp points to float b
char   *cp = &c;    // cp points to character c

// NULL pointer (safe uninitialized pointer)
int *np = NULL;    // np points to nothing (0x0)
```

3.3 Memory Layout of Pointers

FIG 3.1 - POINTER MEMORY ADDRESS DIAGRAM

Variable	Value	Address	Type
a	100	0x2000	int
ip (ptr to a)	0x2000	0x3000	int*
b	3.14	0x2004	float

*ip stores address 0x2000 (address of a). *ip → 100.*

3.4 Null Pointer

A **null pointer** is a pointer that does not point to any memory location. It is represented as `NULL` (value 0). It is a good practice to initialize pointers to `NULL` when not immediately assigned.

Warning: Dereferencing a `NULL` pointer causes a *segmentation fault* (runtime crash). Always check if a pointer is `NULL` before dereferencing it.

Pointer Arithmetic

4.1 Definition

Pointer arithmetic refers to arithmetic operations performed on pointer variables. When arithmetic is applied to a pointer, it moves the pointer by units of the *size of the pointed data type*, not by individual bytes.

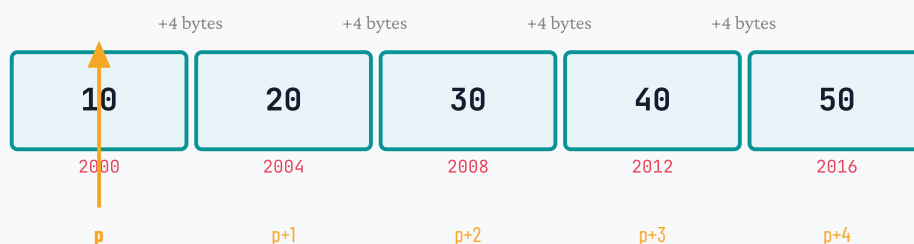
4.2 Valid Pointer Operations

Operation	Description	Example
<code>p++</code>	Increment by <code>sizeof(type)</code>	<code>int *p; p++</code> → moves by 4 bytes
<code>p--</code>	Decrement by <code>sizeof(type)</code>	<code>p--</code> → moves back by 4 bytes
<code>p + n</code>	Move forward <code>n</code> elements	<code>p + 3</code> → $3 \times 4 = 12$ bytes ahead
<code>p - n</code>	Move backward <code>n</code> elements	<code>p - 2</code> → $2 \times 4 = 8$ bytes behind
<code>p1 - p2</code>	Difference in elements	Number of elements between <code>p1</code> and <code>p2</code>
<code>p1 == p2</code>	Compare two pointers	Check if they point to same location

4.3 How Pointer Arithmetic Works

If `p` is an `int *` pointing to address 2000, then `p+1 = 2004` (because `sizeof(int) = 4`), `p+2 = 2008`, etc.

FIG 4.1 - POINTER ARITHMETIC ILLUSTRATION



`int arr[]` stored at 2000. `sizeof(int)=4`. `p+1` jumps to 2004.

```
#include <stdio.h>

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int *p = arr;          // p points to arr[0]

    printf("p    = %p, *p    = %d\n", p, *p);    // 2000, 10
    p++;
    printf("p++ = %p, *p    = %d\n", p, *p);    // 2004, 20
    printf("p+2 = %p, *(p+2) = %d\n", p+2, *(p+2)); // 2012, 40

    return 0;
}
```

Pointers and Arrays

5.1 Relationship Between Pointers and Arrays

In C, an array name itself acts as a **constant pointer** to the first element. The array name `arr` is equivalent to `&arr[0]`. Therefore, pointers and arrays are closely related and largely interchangeable in expression context.

```
arr ≡ &arr[0] (base address)
arr+i ≡ &arr[i] (address of i-th element)
*(arr+i) ≡ arr[i] (value of i-th element)
```

FIG 5.1 - POINTER AND ARRAY RELATIONSHIP DIAGRAM



arr → base address. `arr[i] == *(arr+i)`. Both notations are equivalent.

5.2 Key Points

- **Array name is a constant pointer** — cannot do `arr++` (`arr` is not modifiable)
- Pointer variable pointing to array can be incremented: `int *p = arr; p++;`
- `arr[i]` and `*(arr+i)` are exactly equivalent
- `&arr[i]` and `(arr+i)` give the same address

C PROGRAM — ARRAY ACCESS USING POINTER

```
#include <stdio.h>

int main() {
    int arr[] = {5, 10, 15, 20, 25};
    int *p = arr; // p → arr[0]
```

```
int i;

printf("Using array notation:\n");
for(i = 0; i < 5; i++)
    printf("arr[%d] = %d\n", i, arr[i]);

printf("\nUsing pointer notation:\n");
for(i = 0; i < 5; i++)
    printf("*(p+%d) = %d\n", i, *(p+i));

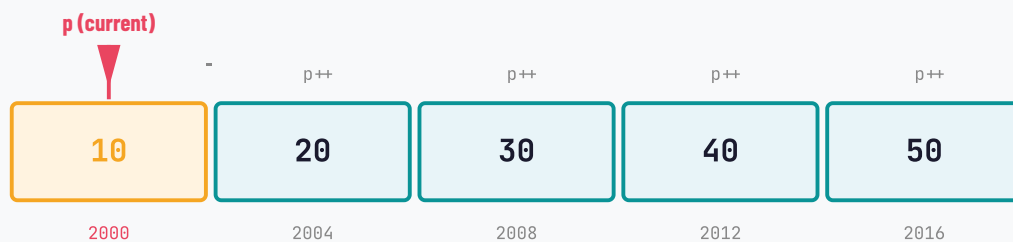
return 0;
}
```

Array Manipulation Using Pointers

6.1 Traversal Using Pointers

A pointer can be used to traverse an array by incrementing it in a loop. This method is more efficient than using subscript notation as it avoids computing the address each time.

FIG 6.1 - ARRAY TRAVERSAL USING POINTER



Loop: `while(p <= end) { print *p; p++; }` — traverses array with pointer

C PROGRAM — ARRAY MANIPULATION USING POINTERS

```
#include <stdio.h>

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = 5, sum = 0;
    int *p = arr;          // p → first element
    int *end = arr + n;    // p pointing past last element

    // Traversal and sum
    while(p < end) {
        sum += *p;
        printf("Address: %p | Value: %d\n", p, *p);
        p++;
    }
    printf("Sum = %d\n", sum);

    // Modify array using pointers
    p = arr;
    for(int i = 0; i < n; i++)
        *(p + i) *= 2;    // double each element

    printf("Doubled: ");
    for(int i = 0; i < n; i++)
```

```
        printf("%d ", arr[i]);

    return 0;
}
```

6.2 Comparison: Array vs Pointer Notation

Operation	Array Notation	Pointer Notation
Access element i	arr[i]	*(p+i) or *(arr+i)
Address of element i	&arr[i]	p+i or arr+i
Traverse forward	index++	p++
Array name modifiable?	No (constant pointer)	Yes (p is a variable)
Size calculation	sizeof(arr)	Not directly available

Introduction to User Defined Data Types

7.1 What are User Defined Data Types?

C provides primitive data types like `int`, `float`, `char`. However, real-world data is often complex and grouped. **User Defined Data Types (UDTs)** allow programmers to define their own data types to represent complex, structured data.

7.2 Types of User Defined Data Types in C

Type	Keyword	Description
Structure	<code>struct</code>	Groups variables of different types under one name. Each member has its own memory.
Union	<code>union</code>	Groups variables sharing the same memory location. Only one member holds valid data at a time.
Enumeration	<code>enum</code>	Defines a set of named integer constants.
Typedef	<code>typedef</code>	Creates an alias for existing data types.

7.3 Why UDTs?

- Organize related data logically (e.g., student: name, age, marks)
- Make code more readable and maintainable
- Enable creation of real-world data models
- Foundation of object-oriented concepts

Real-World Analogy: A *Student Record* in a college database has name (string), age (int), and GPA (float). In C, a structure groups all these into one unit — just like a form.

Structures in C

8.1 Definition

A **structure** is a user-defined data type in C that groups related variables of *different data types* under a single name. Each variable inside a structure is called a **member** or **field**.

8.2 Structure Declaration Syntax

```
struct tag_name {  
    data_type member1;  
    data_type member2;  
    ...  
};  
  
// Declaring a structure variable:  
struct tag_name variable_name;
```

8.3 Structure Declaration and Initialization

STRUCTURE DECLARATION, INITIALIZATION, AND ACCESS

```
#include <stdio.h>  
#include <string.h>  
  
struct Student {  
    int    rollNo;  
    char   name[50];  
    float  marks;  
};  
  
int main() {  
    // Method 1: Individual assignment  
    struct Student s1;  
    s1.rollNo = 101;  
    strcpy(s1.name, "Arjun");  
    s1.marks = 95.5;  
  
    // Method 2: Initializer list  
    struct Student s2 = {102, "Priya", 88.0};  
  
    printf("Roll: %d, Name: %s, Marks: %.1f\n",  
           s1.rollNo, s1.name, s1.marks);  
    printf("Roll: %d, Name: %s, Marks: %.1f\n",
```



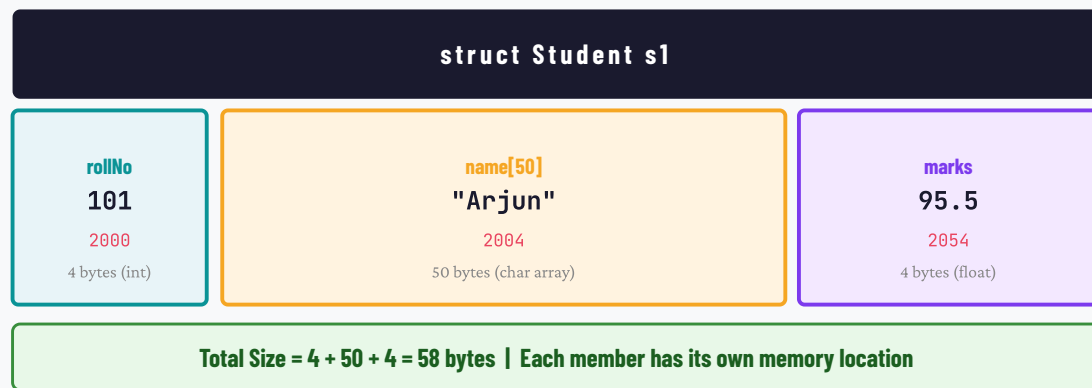
```
s2.rollNo, s2.name, s2.marks);
```

```
return 0;
```

```
}
```

8.4 Structure Memory Layout

FIG 8.1 - STRUCTURE MEMORY LAYOUT DIAGRAM



Members are stored in contiguous memory; total = sum of all member sizes (+ padding).

8.5 Accessing Structure Members

Structure members are accessed using the **dot (.) operator** for normal variables and the **arrow (->) operator** for pointers to structures.

```
variable.member → for structure variable  
pointer→member → for pointer to structure  
(*pointer).member → equivalent to pointer→member
```

Structure Operations

9.1 Nested Structures

A structure can contain another structure as a member. This is called a **nested structure**. It is used to model hierarchical data.

NESTED STRUCTURE EXAMPLE

```
struct Date {
    int day, month, year;
};

struct Employee {
    int empId;
    char name[50];
    struct Date joinDate;    // nested structure
};

int main() {
    struct Employee e1;
    e1.empId = 501;
    strcpy(e1.name, "Rahul");
    e1.joinDate.day    = 15;
    e1.joinDate.month  = 8;
    e1.joinDate.year   = 2023;

    printf("Emp: %s joined on %d/%d/%d\n",
           e1.name, e1.joinDate.day,
           e1.joinDate.month, e1.joinDate.year);
    return 0;
}
```

FIG 9.1 - NESTED STRUCTURE REPRESENTATION

struct Employee

empld
501

name[50]
"Rahul"

struct Date (joinDate)

day
15

month
8

year
2023

Access: e1.joinDate.day, e1.joinDate.month, e1.joinDate.year

9.2 Pointer to Structure

POINTER TO STRUCTURE USING ARROW OPERATOR

```
struct Student s = {101, "Anika", 90.5};  
struct Student *ptr = &s;  
  
// Both are equivalent:  
printf("%d\n", (*ptr).rollNo); // using *  
printf("%d\n", ptr->rollNo);   // using → (preferred)  
printf("%s\n", ptr->name);  
printf("%.1f\n", ptr->marks);
```

Arrays of Structures

10.1 Definition

An **array of structures** allows storing multiple structure variables of the same type in a contiguous block of memory. It is used to represent collections of records (e.g., list of students, employee database).

Syntax: `struct tag_name array_name[size];`

Example: `struct Student s[100];` // Array of 100 Student records

FIG 10.1 - ARRAY OF STRUCTURES MEMORY DIAGRAM



Each structure occupies 58 bytes. $s[1]$ starts at $2000+58=2058$.

C PROGRAM – ARRAY OF STRUCTURES

```
#include <stdio.h>

struct Student {
    int    rollNo;
    char   name[30];
    float  marks;
};

int main() {
    struct Student s[3] = {
        {101, "Arjun", 95.5},
        {102, "Priya", 88.0},
        {103, "Ravi", 91.5}
    };

    printf("%-6s %-15s %s\n", "Roll", "Name", "Marks");
```

```
printf("-----\n");

for(int i = 0; i < 3; i++) {
    printf("%-6d %-15s %.1f\n",
        s[i].rollNo, s[i].name, s[i].marks);
}

return 0;
}
```

Unions in C

11.1 Definition

A **union** is a user-defined data type where all members *share the same memory location*. The size of a union is equal to the size of its *largest member*. Only one member can hold a valid value at any given time.

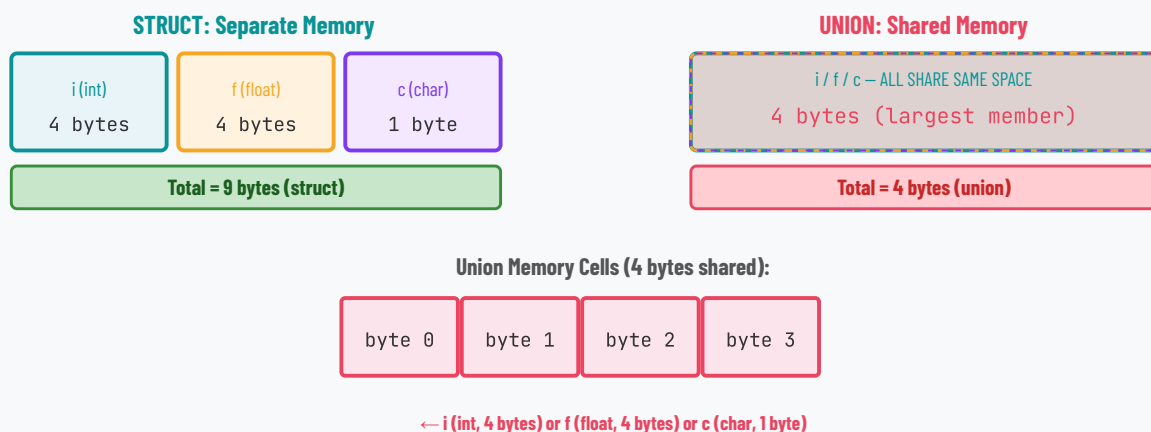
Syntax:

```
union tag_name {
    data_type member1;
    data_type member2;
};

union tag_name variable_name;
```

11.2 Union Memory Sharing

FIG 11.1 – UNION MEMORY SHARING DIAGRAM



Only one member can be active/valid at a time. Assigning a new member overwrites previous.

C PROGRAM – UNION EXAMPLE

```
#include <stdio.h>

union Data {
    int i;
    float f;
    char c;
```

```
};

int main() {
    union Data d;
    printf("Size of union: %lu bytes\n", sizeof(d)); // 4

    d.i = 100;
    printf("d.i = %d\n", d.i); // 100 ✓

    d.f = 3.14;
    printf("d.f = %f\n", d.f); // 3.14 ✓
    printf("d.i = %d\n", d.i); // corrupted! f overwrote i

    d.c = 'A';
    printf("d.c = %c\n", d.c); // A ✓

    return 0;
}
```

11.3 Applications of Unions

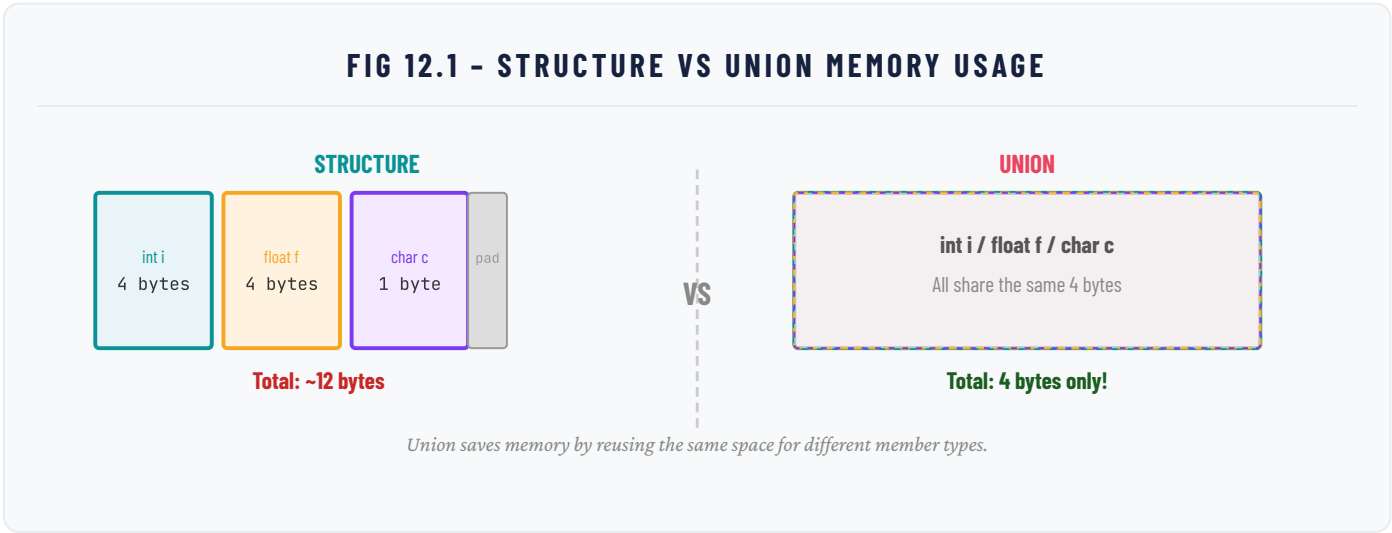
- **Memory-sensitive embedded systems** where every byte counts
- **Type punning:** Interpret same bytes as different types
- **Protocol parsing:** Parse network packets efficiently
- **Variant records:** Store different data types in same field

Structure vs Union

12.1 Key Differences Table

Feature	Structure	Union
Keyword	struct	union
Memory Allocation	Separate memory for each member	Shared memory — all members use same space
Size	Sum of sizes of all members (+ padding)	Size of the largest member
Member Access	All members can be used simultaneously	Only one member valid at a time
Data Integrity	All member values preserved	Overwriting one member corrupts others
Memory Efficiency	Less efficient (more memory used)	More efficient (minimum memory used)
Use Case	Grouped heterogeneous data (student, employee records)	Variant data types, embedded systems, protocol parsing
Initialization	All members can be initialized	Only first member can be initialized at declaration
Example Size	struct{int i; float f; char c;} = 9+ bytes	union{int i; float f; char c;} = 4 bytes

12.2 Memory Comparison Diagram



12.3 When to Use What?

Scenario	Use
Storing multiple attributes of an entity (student, employee)	Structure
Embedded systems with memory constraints	Union
All member data must be available simultaneously	Structure
Only one field is active at a time	Union
Database records, file I/O, API data	Structure
Network packet parsing, type conversion	Union

Questions & Answers

2-Mark Questions

2 MARKS

Q1. What is a pointer in C? Give its syntax.

ANSWER

A **pointer** is a variable that stores the memory address of another variable. It is declared using the asterisk (*) symbol before the variable name. The data type of the pointer determines the type of variable it can point to.

Syntax: `data_type *pointer_name;`

Example: `int a = 10; int *p = &a;` — here p holds the address of a. Accessing *p gives the value 10. Pointers enable dynamic memory management and efficient array traversal in C.

2 MARKS

Q2. Differentiate between & and * operators in pointers.

ANSWER

The **& (address-of) operator** returns the memory address of a variable. Example: `&a` gives the address (say 2000) where variable a is stored.

The *** (dereference) operator** accesses the value at the address stored in a pointer. Example: if `p = &a`, then `*p` returns 10 (the value of a).

In short: `&` goes from variable to address; `*` goes from address to value. They are inverse operations in pointer usage.

2 MARKS

Q3. What is a structure? How is it declared?

ANSWER

A **structure** is a user-defined data type in C that groups variables of different data types under a single name. Each variable inside is called a member. Structures are used to represent real-world entities with multiple attributes.

Declaration: `struct Student { int roll; char name[50]; float marks; };
struct Student s1;`

Members are accessed using the dot (.) operator: `s1.roll = 101;` Structures allocate separate memory for each member, so all members can hold values simultaneously.

2 MARKS

Q4. What is the size of a union? Explain with an example.

ANSWER

The size of a **union** equals the size of its largest member. This is because all members of a union share the same memory location.

Example: `union U { int i; float f; char c; };` — `sizeof(int)=4`, `sizeof(float)=4`, `sizeof(char)=1`. So `sizeof(U) = 4 bytes` (size of largest member).

Compare: a struct with same members would be 9+ bytes. Union saves memory at the cost of only allowing one active member at a time. Assigning a new member overwrites any previously stored data.

5-Mark Questions

Q5. Explain pointers in C with a detailed example. Include memory representation.**ANSWER**

A **pointer** is a variable that contains the address of another variable in memory. Pointers are fundamental to C programming because they allow direct memory access and manipulation.

Declaration: `int *p;` — declares a pointer to integer. The data type tells how many bytes to read when dereferencing.

Initialization: `int a = 10; p = &a;` — p now holds the address of a (e.g., 2000).

Dereferencing: `*p` reads the value at address stored in p, giving 10. Writing `*p = 50` changes a to 50.

Memory: Variable a is at address 2000 with value 10. Pointer p is at address 3000 with value 2000 (address of a).

Pointer Arithmetic: If p = 2000, then p+1 = 2004 for int pointer (moves by sizeof(int)=4 bytes).

Applications: Pointers are used in dynamic memory allocation, passing arrays to functions, building linked lists, and accessing hardware addresses in embedded systems.

5-MARK PROGRAM – POINTERS WITH MEMORY

```
#include <stdio.h>

int main() {
    int a = 10, b = 20;
    int *p = &a;

    printf("a = %d, address = %p\n", a, &a);
    printf("*p = %d, p = %p\n", *p, p);

    *p = 50;    // modifies a
    printf("After *p=50: a = %d\n", a);

    p = &b;    // now p points to b
    printf("Now *p = %d (b's value)\n", *p);

    return 0;
}
```

Q6. Explain pointer arithmetic with examples and a program.**ANSWER**

Pointer arithmetic allows numeric operations on pointers. The key rule is: the pointer moves by multiples of the *size of the data type* it points to, not by bytes.

Rules of Pointer Arithmetic:

- **Increment (p++):** Moves pointer forward by sizeof(type). For int*, p++ adds 4 bytes.
- **Decrement (p--):** Moves pointer backward by sizeof(type).
- **Addition (p+n):** Moves pointer n elements forward.
- **Subtraction (p-n):** Moves pointer n elements backward.
- **Difference (p1-p2):** Returns number of elements between p1 and p2.
- **Comparison:** Pointers can be compared using ==, !=, <, > when pointing to same array.

Example: int *p at address 2000. p+1 = 2004, p+2 = 2008, p+3 = 2012. Each step adds sizeof(int) = 4 bytes.

POINTER ARITHMETIC – COMPLETE PROGRAM

```
#include <stdio.h>

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int *p = arr;

    printf("Base address: %p\n", p);

    for(int i = 0; i < 5; i++) {
        printf("p+%d = %p → value = %d\n",
              i, p+i, *(p+i));
    }

    // Pointer difference
    int *p1 = &arr[0];
    int *p2 = &arr[4];
    printf("Difference: %ld elements\n", p2 - p1); // 4

    return 0;
}
```

Q7. Write a C program to manipulate an array using pointers.**ANSWER**

Arrays and pointers are closely related in C. The array name is a constant pointer to the first element. We can use a pointer to traverse, read, and modify array elements efficiently.

Key Equivalences:

- `arr[i] == *(arr+i) == *(p+i)`
- `&arr[i] == arr+i == p+i`

Using pointer traversal is efficient because the processor directly increments the pointer register without computing base + offset each time.

ARRAY MANIPULATION USING POINTERS – COMPLETE PROGRAM

```
#include <stdio.h>

void printArray(int *p, int n) {
    for(int i = 0; i < n; i++)
        printf("%d ", *(p + i));
    printf("\n");
}

int main() {
    int arr[] = {5, 3, 8, 1, 9};
    int n = 5;
    int *p = arr;

    printf("Original: ");
    printArray(p, n);

    // Find sum using pointer
    int sum = 0;
    for(int i = 0; i < n; i++)
        sum += *(p + i);
    printf("Sum = %d\n", sum);

    // Find max using pointer
    int max = *p;
    for(int i = 1; i < n; i++)
        if(*(p+i) > max)
            max = *(p+i);
    printf("Max = %d\n", max);

    // Reverse using pointer
    int *left = p;
    int *right = p + n - 1;
    while(left < right) {
```

```
        int temp = *left;
        *left++ = *right;
        *right-- = temp;
    }
    printf("Reversed: ");
    printArray(arr, n);

    return 0;
}
```

Q8. Explain structures in C with declaration, initialization, and a complete example program.

ANSWER

A **structure** (struct) in C is a user-defined data type that groups related variables of potentially different data types under a single name. Each variable in a structure is called a *member*.

Declaration: Defined using the `struct` keyword followed by a tag name and a list of member declarations inside braces.

Initialization: A structure variable can be initialized at declaration using an initializer list, or members can be assigned individually using the dot operator.

Memory: Each member gets its own memory block. Total size is the sum of all member sizes (plus possible padding added by the compiler for alignment).

Member Access: Dot (.) operator for variables, arrow (->) operator for pointers to structures.

Nested Structures: A structure can contain another structure as a member, enabling hierarchical data modeling (e.g., Employee containing Date structure).

STRUCTURE – COMPLETE PROGRAM WITH ALL OPERATIONS

```
#include <stdio.h>
#include <string.h>

struct Student {
    int    roll;
    char   name[30];
    float  marks;
    char   grade;
};

char calcGrade(float m) {
    if(m ≥ 90) return 'A';
    if(m ≥ 80) return 'B';
    if(m ≥ 70) return 'C';
    return 'D';
}

int main() {
    struct Student s[3] = {
        {101, "Arjun", 95.0},
        {102, "Priya", 82.5},
        {103, "Ravi", 75.0}
    };
};
```



```
for(int i = 0; i < 3; i++)
    s[i].grade = calcGrade(s[i].marks);

printf("%-5s %-15s %-8s %s\n",
        "Roll", "Name", "Marks", "Grade");
for(int i = 0; i < 3; i++) {
    printf("%-5d %-15s %-8.1f %c\n",
            s[i].roll, s[i].name,
            s[i].marks, s[i].grade);
}
return 0;
}
```

Q9. Differentiate between structure and union with examples. (Most Important)**ANSWER**

Both `struct` and `union` are user-defined data types in C used to group variables. However, they differ fundamentally in how they allocate memory.

Key Difference: In a structure, each member has its own memory space. In a union, all members share the same memory space — equal to the size of the largest member.

Structure Size: Sum of all member sizes (+ alignment padding). A struct `{int i; float f; char c;}` = approximately 12 bytes.

Union Size: Size of largest member only. A union `{int i; float f; char c;}` = 4 bytes (`sizeof int` or `float`, whichever is largest).

Data preservation: In structures, all members hold their values simultaneously. In unions, assigning one member overwrites (corrupts) the others.

Use Case: Structures are used for student/employee records where all fields are needed together. Unions are used in embedded systems, protocol parsing, and memory-constrained applications.

STRUCTURE VS UNION – SIDE-BY-SIDE PROGRAM

```
#include <stdio.h>

struct SData { int i; float f; char c; };
union UData { int i; float f; char c; };

int main() {
    struct SData s;
    s.i = 10; s.f = 3.14; s.c = 'A';
    printf("STRUCT size = %lu\n", sizeof(s));
    printf("s.i=%d s.f=%.2f s.c=%c\n",
           s.i, s.f, s.c);           // all valid

    union UData u;
    u.i = 10;
    printf("\nUNION size = %lu\n", sizeof(u));
    printf("u.i = %d\n", u.i);       // valid
    u.f = 3.14;
    printf("u.f = %.2f\n", u.f);     // valid
    printf("u.i = %d\n", u.i);       // CORRUPTED!
```

```
return 0;
```

```
}
```

Quick Revision Sheet & References

14.1 Quick Revision Sheet

POINTER SYNTAX SUMMARY

- `int *p;` — declare pointer
- `p = &a;` — assign address
- `*p` — dereference (get value)
- `p++` — next element (+4 for int)
- `p+n` — n elements ahead
- `NULL` — empty/safe pointer
- `int **pp` — pointer to pointer

POINTER ARITHMETIC TABLE

- `int* p++:` +4 bytes
- `float* p++:` +4 bytes
- `double* p++:` +8 bytes
- `char* p++:` +1 byte
- `p1 - p2:` number of elements between
- `*(p+i) == arr[i]`
- `p+i == &arr[i]`

STRUCTURE KEY POINTS

- Keyword: `struct`
- Each member → own memory
- Size = sum of members + padding
- Access: `variable.member`
- Pointer: `ptr->member`
- All members valid simultaneously
- Used for: records, entities

UNION KEY POINTS

- Keyword: `union`
- All members → same memory
- Size = largest member size
- Access: same as struct (`.` and `->`)
- Only ONE member valid at a time
- Used for: memory efficiency
- Embedded systems, protocol parsing

14.2 Structure vs Union – One-Line Comparison

Property	struct	union
Memory	Separate per member	Shared (largest member size)
Size	Sum of all members	Size of largest member
Simultaneous access	Yes — all valid	No — only one valid
Use case	Complex data records	Memory-saving, variant data

14.3 Important Programs Quick Reference

POINTER TO ARRAY

- `int arr[5]; int *p = arr;`
- `*(p+i)` to access `arr[i]`
- `while(p < arr+n) { *p++; }`
- Sum, max, reverse using p

ARRAY OF STRUCTURES

- `struct S arr[n];`
- `for(i=0;i<n;i++) arr[i].member`
- Initialize: {val1, val2, ...}
- Sort, search through array

14.4 Memory Concepts Summary

Key Memory Rules to Remember:

1. Every variable has a unique address in memory.
2. Pointer variable itself takes 4 (32-bit) or 8 (64-bit) bytes regardless of type.
3. `p++` for `int*` = +4 bytes; for `char*` = +1 byte; for `double*` = +8 bytes.
4. struct size = sum of members + compiler padding for alignment.
5. union size = size of largest member only.
6. Array name = constant pointer to first element. Cannot do `arr++`.
7. NULL pointer (0x0) — never dereference; causes segmentation fault.

14.5 Textbook References

PRIMARY TEXTBOOKS

The C Programming Language (2nd Ed.) — Brian W. Kernighan & Dennis M. Ritchie, Prentice Hall, 1988. ISBN: 0-13-110362-8. The definitive reference for C language including pointers, structures, and memory management.

Schaum's Outline of Programming with C — Byron S. Gottfried, McGraw-Hill, 1996. Comprehensive outline with solved problems covering all C concepts from basic to advanced.

REFERENCE BOOKS

Computing Fundamentals and C Programming — E. Balagurusamy, McGraw-Hill Education, 2008. Widely used undergraduate textbook covering C fundamentals with examples and exercises.

Programming in C — Rema Theraja, Oxford University Press, 2016. Clear explanations of pointers and user-defined data types with practical examples.

C Programming: A Problem Solving Approach (3rd Ed.) — Forouzan, Gilberg & Prasad, Cengage Learning. Problem-oriented approach to C with coverage of pointers, structures, and unions with real-world applications.

UNIT IV – COMPLETE

Pointers & User Defined Data Types — B.Tech I Year, C Programming

14 Sections · 10 Diagrams · 12 Programs · 9 Q&A